

Online Fee Management System

Project submitted to the APSSDC

Submitted By

N. N. V.R. Manohar

B. Kumar

I. Srinivasu

K. Shanmuka Sai Vignesh

D. Narendra



Under the guidance of

B. Sumana

J. Lova Babu

TABLE OF CONTENTS

S.NO	CONTENTS	PAGE NO
1	Abstract	3
2	Introduction	4
3	System Requirements	5
4	Architecture	6
5	AWS Services	7
6	Git Hub	8
7	Project Code	9-16
8	Git hub pages	17
9	AWS pages	18-19
10	Conclusion	20

ABSTRACT

This study presents a detailed report of Online Fee Management System is a web-based solution designed to simplify and automate the complex processes of fee collection and record management for educational institutions. Built using Django, a powerful Python - based web framework, the system provides a secure, scalable, and user-friendly platform for administrators, students, and staff to manage financial transactions digitally.

The primary objectives of this System are as follows:

- **Student registration & Profile Management:** This involves collecting and managing student data such as personal information, academic history, contact details, etc. It also includes processes like storing, retrieving, and updating student data related to registration, academic records, and other relevant information associated with registration.
- **Real-time Fee Submission:** Real-time fee submission enables students to pay fees instantly through the system, eliminating the need for manual processing and reducing delays.
- **Dynamic Fee Structure Configuration:** Dynamic fee structure configuration in online fee management systems empowers institutions to manage fees efficiently, accurately, and transparently, while also providing flexibility to meet the evolving needs of their students and stakeholders.
- **Role-Based Access Control for Admins, Accountant, and Students:** Different roles, like administrators and accountants, have distinct responsibilities. Role based access control ensures that only authorized individuals can perform specific actions, preventing fraud and errors and provide secure environment for students to pay fees.
- **Detailed Transaction History:** Detailed transaction history is not just a record of payments; it's a tool that enhances transparency, accountability, and efficiency in the online fee management system.

Throughout this project, Django-based online fee management system learning outcome would include proficiency in using Django framework, database management , and implementing features like user authentication, fee calculation, invoice generation, online payment integration, and report generation. Students would also gain experience in building a web application with a clear separation of concerns, using models, views, and templates.

By successfully completing this project, we gain a solid foundation in Django web development, database management, and the practical skills needed to build a real-world online fee management system.

INTRODUCTION

In the digital age, educational institutions are rapidly transitioning from manual administrative processes to automated systems to enhance efficiency and transparency. One such critical transformation is the adoption of an Online Fee Management System, a web-based solution designed to streamline the entire fee collection and management process.

Traditionally, fee handling involved extensive paperwork, long queues, and a high risk of human error. The online system eliminates these challenges by offering a centralized, secure, and user-friendly platform for administrators, students. It automates tasks such as invoice generation, payment tracking, significantly reducing administrative workload and improving financial accuracy.

By embracing this system, institutions not only enhance operational efficiency but also foster trust and convenience among stakeholders. It's a step toward smarter, more transparent education management.

Key benefits include:

- Real-time payment tracking
- Integration with digital payment gateways for secure transactions
- Customizable fee structure based on branch.
- Comprehensive financial reports for better decision-making

An Online Fee Management System addresses the inefficiencies of traditional fee collection methods. Here's a concise analysis:

- **Problem Statement:** Manual fee processing is time-consuming, error-prone, and lacks transparency.
- **Solution:** A web-based system that automates fee collection, receipt generation, and reporting.
- **Technology Stack:** Typically built using Django (Python), HTML/CSS for frontend, and a relational database like dbsqlite.

The Online Fee Management System is a secure, web-based platform designed to simplify and automate the entire fee collection process for educational institutions. Key features include student profile and customizable fee structure management, integration with digital payment gateways, real-time tracking, automated reminders, and generation of accurate financial reports. Through its development, we gain hands-on experience with Django's web-based application.

SYSTEM REQUIREMENTS

Operating System

- Windows 10/11 (64-bit)
- macOS 10.15 or later
- Linux (Ubuntu 18.04+ recommended)

Hardware Requirements

- Processor: Intel i5 / AMD Ryzen 5 or higher
- RAM: Minimum 4 GB (8 GB recommended)
- Storage: Minimum 500 MB free disk space
- Display: 1024×768 resolution or higher

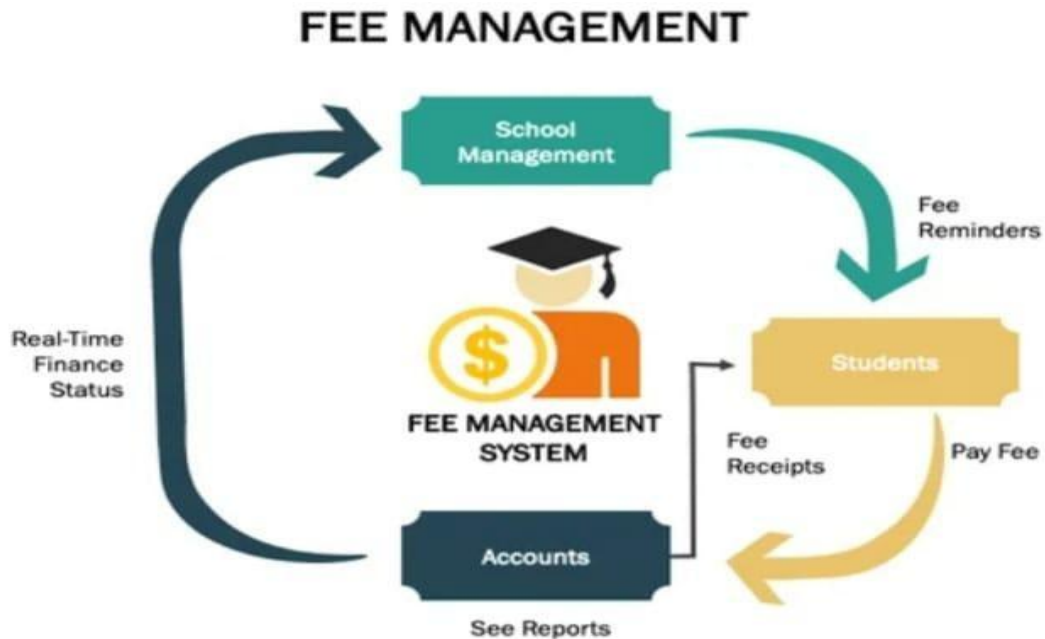
Software Requirements

- Python version: 3.6.8
- Required Python web Framework:
 - Django version 3.2.25
- Development Environment (any one):
 - Jupyter Notebook (via Anaconda or JupyterLab)
 - Google Colab (browser-based)
 - Sublime Text Editor (or) VS Code
 - urls.py
 - views.py
 - models.py
 - forms.py
 - admin.py
 - Command Prompt

Technology Stack

- Frontend: HTML5, CSS, Bootstrap
- Backend: Django Framework (Python)
- Database: MySQL

ARCHITECTURE



An Online Fee Management System built with Django streamlines the entire process of collecting and managing student fees through a web-based interface. The process typically begins with administrators setting up fee structures in the system—such as tuition, transport, or hostel fees—based on different classes or academic programs. Students or parents can then log in through a secure portal to view their assigned fee details and make payments using integrated payment gateways. Once a payment is completed, the system instantly updates the transaction record and generates a digital receipt, ensuring transparency and accuracy. Meanwhile, administrative staff can monitor all incoming payments in real time, track pending or overdue fees, and use built-in filters to generate reports by date, student, payment status. Throughout the process, Django’s role-based access control ensures that sensitive financial data is only accessible to authorized users like administrators and accountants. This structured flow—from fee assignment to payment and reporting—makes the system highly efficient, secure, and adaptable to the needs of any educational institution.

USES OF PYTHON FILES

- **Urls.py – Uniform Resource Locator**

- This file acts as the URL dispatcher.
- It maps specific URL patterns to corresponding view functions.
- Without this, Django wouldn't know what to do when a user visits a specific page.
- Connects URLs
- Helps modularize routing by including app-specific URLs.

- **Views.py**

- View functions handle incoming requests, process data , and return responses.
- Fetches data from the database using models
- Renders templates with context data
- Handles form submissions and redirects
- It's the bridge between data (models) and what the user sees (templates).

- **Models.py**

- This file defines database models .
- Each model maps to a database table.
- Each class attribute becomes a database field
- Automatically creates tables and handles queries
- It's our system's memory—where all fee records, student info, and transactions are stored.

- **Forms.py**

- This file defines form classes that handle user input, validation, and rendering.
- Creates forms like 'FeePaymentForm', 'StudentRegistrationForm'
- Validates input (e.g., required fields, correct formats)
- Can be based on models ('ModelForm')
- It ensures that data entered by users is clean, valid, and ready to be saved or processed.

- **Admin.py**

- Model Registration: Connects models like 'Student', 'Fee', and 'Payment' to the Django admin dashboard.
- Custom Admin Interfaces: Enhances usability with to organize fee and student data.
- Quick Data Management: Admin users can update fee status, view payments, and track dues without needing a custom UI.

AWS Services Used

The Cloud-Based Student Fee Management System is deployed using three core Amazon Web Services (AWS): Amazon EC2, Amazon RDS, and AWS IAM. These services provide computing power, secure database management, and access control, enabling the application to run efficiently and securely on the cloud.

1. Amazon EC2 (Elastic Compute Cloud)

Amazon EC2 is a cloud computing service that provides virtual servers for hosting applications. In this project, an Ubuntu-based EC2 instance is used to deploy and run the Django web application. The application is configured on the EC2 instance using Python, Django, and the required dependencies. Users access the Student Fee Management System through the EC2 instance's public IP address. EC2 offers flexibility, scalability, and cost-effective computing resources, making it suitable for hosting web applications.

2. Amazon RDS (Relational Database Service)

Amazon RDS is a managed relational database service that simplifies database administration. Instead of using SQLite, the project uses Amazon RDS MySQL to store student records, accountant details, fee information, and user authentication data. The Django application connects securely to the RDS instance using the database endpoint, username, password, and port number. RDS provides features such as automated backups, high availability, improved performance, and secure data storage, making it more reliable than a local database.

3. AWS IAM (Identity and Access Management)

AWS IAM is used to securely manage access to AWS resources. For this project, an IAM user named manohar-dev was created instead of using the AWS root account for daily operations. The IAM user was added to a Developers group with the required permissions to manage AWS resources. IAM improves the security of the project by implementing role-based access control and following the principle of least privilege. This ensures that only authorized users can access or modify cloud resources.

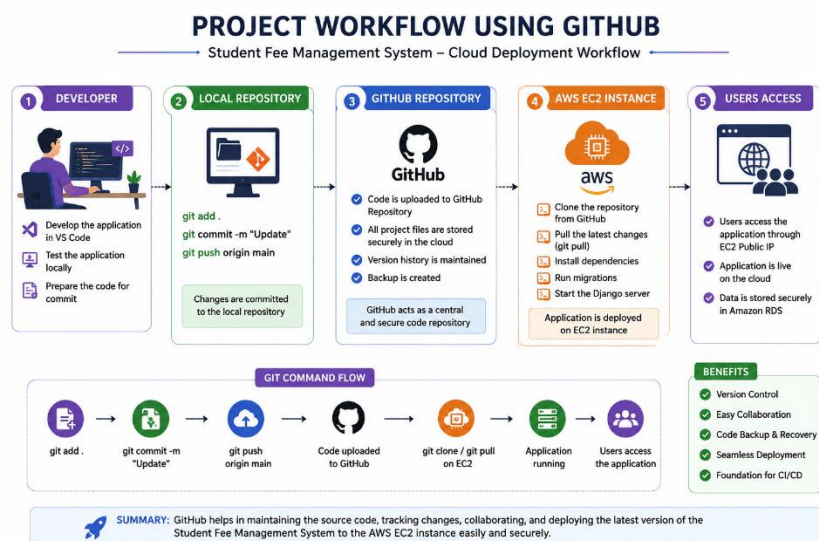
GitHub in the Student Fee Management System

GitHub is a cloud-based version control and code hosting platform that enables developers to store, manage, and track changes to their source code using Git. It provides a centralized repository where project files are securely maintained, making software development more organized and reliable.

In the Cloud-Based Student Fee Management System, GitHub is used to store the complete Django project source code. After developing the application locally using Visual Studio Code, the project is uploaded to a GitHub repository. The AWS EC2 instance then clones the repository and deploys the application to the cloud. Whenever changes are made, the updated code is pushed to GitHub and pulled into the EC2 instance, ensuring the latest version of the application is always deployed.

Benefits of GitHub in This Project

- Stores the project source code securely in the cloud.
- Maintains version history of all code changes.
- Acts as a backup and recovery solution for the project.
- Simplifies deployment from GitHub to AWS EC2.
- Enables easy updates using Git Push and Git Pull.
- Supports team collaboration and future DevOps automation (CI/CD).

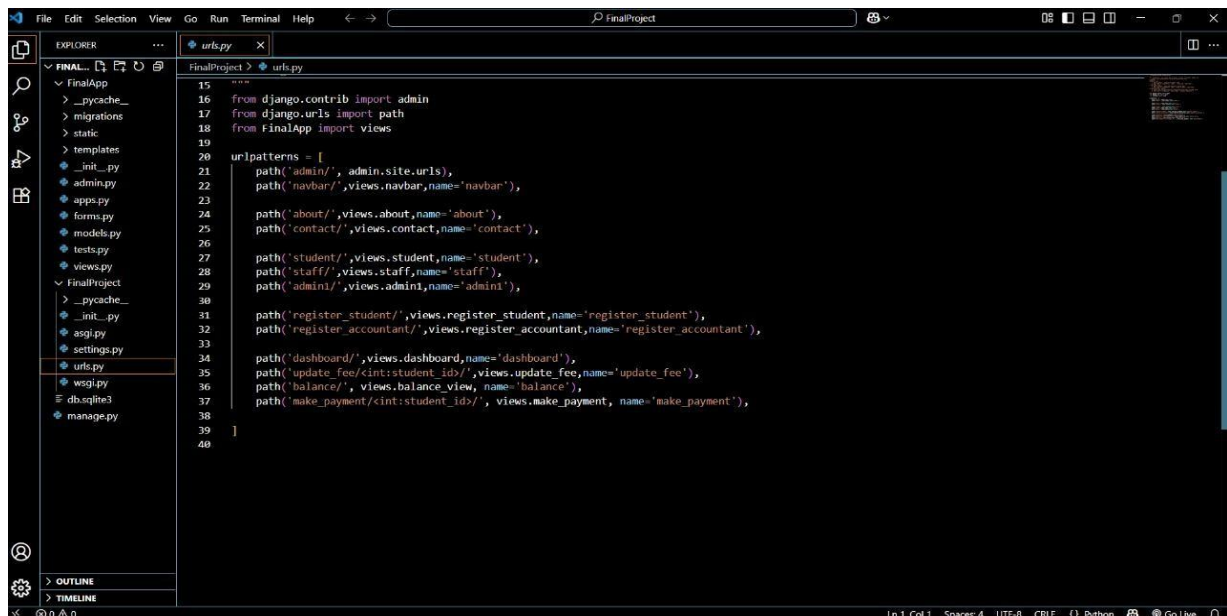


PROJECT CODE

Creating the Environment to Code

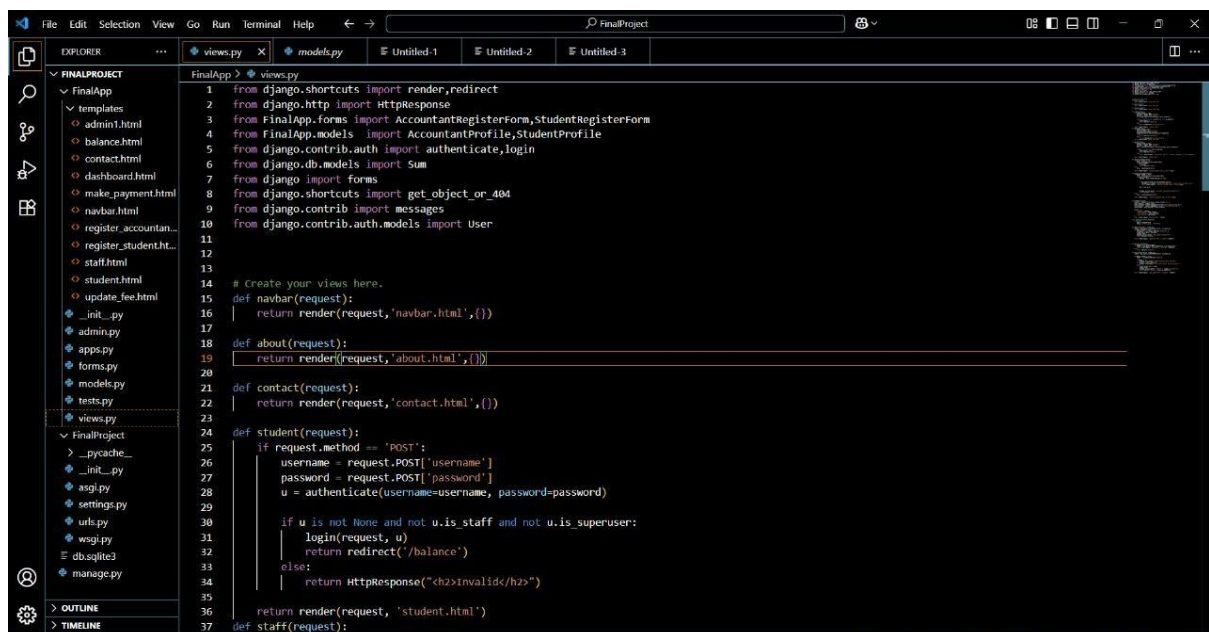
1. Create a folder in desktop and name as Final Project
2. Open the file in cmd prompt
3. To create any project in Django we use the command `django-admin startproject projectname`
4. For the url creation Django provides one python file i.e `urls.py`
5. Likewise `Views.py`, `Models.py`, `Forms.py`, `Admin.py` are also created.

Urls.py



```
15 from django.contrib import admin
16 from django.urls import path
17 from django.urls import path
18 from FinalApp import views
19
20 urlpatterns = [
21     path('admin/', admin.site.urls),
22     path('navbar/', views.navbar, name='navbar'),
23
24     path('about/', views.about, name='about'),
25     path('contact/', views.contact, name='contact'),
26
27     path('student/', views.student, name='student'),
28     path('staff/', views.staff, name='staff'),
29     path('admin/', views.admin, name='admin'),
30
31     path('register_student/', views.register_student, name='register_student'),
32     path('register_accountant/', views.register_accountant, name='register_accountant'),
33
34     path('dashboard/', views.dashboard, name='dashboard'),
35     path('update_fee/<int:student_id>', views.update_fee, name='update_fee'),
36     path('balance/', views.balance_view, name='balance'),
37     path('make_payment/<int:student_id>', views.make_payment, name='make_payment'),
38
39 ]
40
```

Views.py



```
1 from django.shortcuts import render, redirect
2 from django.http import HttpResponseRedirect
3 from FinalApp.forms import AccountantRegisterForm, StudentRegisterForm
4 from FinalApp.models import AccountantProfile, StudentProfile
5 from django.contrib.auth import authenticate, login
6 from django.db.models import Sum
7 from django import forms
8 from django.shortcuts import get_object_or_404
9 from django.contrib import messages
10 from django.contrib.auth.models import User
11
12 # Create your views here.
13
14 def navbar(request):
15     return render(request, 'navbar.html', {})
16
17 def about(request):
18     return render(request, 'about.html', {})
19
20 def contact(request):
21     return render(request, 'contact.html', {})
22
23 def student(request):
24     if request.method == 'POST':
25         username = request.POST['username']
26         password = request.POST['password']
27         u = authenticate(username=username, password=password)
28
29         if u is not None and not u.is_staff and not u.is_superuser:
30             login(request, u)
31             return redirect('/balance')
32         else:
33             return HttpResponseRedirect("/?<h2>Invalid/h2>")
34
35     return render(request, 'student.html')
36
37 def staff(request):
```

```

FinalProject > FinalApp > views.py
37 def staff(request):
38     if request.method == 'POST':
39         username = request.POST['username']
40         password = request.POST['password']
41         u = authenticate(username=username, password=password)
42         if u:
43             return redirect('/dashboard')
44         else:
45             return HttpResponseRedirect("ch2>Invalid/h2>")
46     return render(request, 'staff.html', {})
47
48 def admin(request):
49     if request.method == 'POST':
50         username = request.POST['username']
51         password = request.POST['password']
52         user = authenticate(request, username=username, password=password)
53
54         if user is not None and user.is_superuser:
55             login(request, user)
56             return redirect('/admin/')
57         else:
58             return render(request, 'admin.html', {'error': 'Invalid credentials or not an admin'})
59     return render(request, 'admin.html')
60
61
62 def register_student(request):
63     if request.method == 'POST':
64         form = StudentRegisterForm(request.POST)
65         if form.is_valid():
66             form.save()
67             return redirect('student')
68         else:
69             form = StudentRegisterForm()
70
71     return render(request, 'register_student.html', {'form': form})
72
73 def register_accountant(request):

```

```

FinalProject > FinalApp > views.py
72
73 def register_accountant(request):
74     if request.method == 'POST':
75         form = AccountantRegisterForm(request.POST)
76         if form.is_valid():
77             username = form.cleaned_data.get('username')
78
79             if User.objects.filter(username=username).exists():
80                 messages.error(request, "Username already exists. Please choose a different one.")
81                 return render(request, 'register_accountant.html', {'form': form})
82             user = form.save()
83
84             messages.success(request, "Accountant registered successfully!")
85             return redirect('/staff/')
86         else:
87             form = AccountantRegisterForm()
88
89     return render(request, 'register_accountant.html', {'form': form})
90
91
92 def dashboard(request):
93     students = StudentProfile.objects.select_related('user')
94     total_students = students.count()
95     total_collected = students.aggregate(total_sum=('amount_paid'))['total'] or 0
96     total_expected = students.aggregate(total_sum=('total_fee'))['total'] or 0
97     total_balance = total_expected - total_collected
98
99     context = {
100         'students': students,
101         'total_students': total_students,
102         'total_collected': total_collected,
103         'total_balance': total_balance
104     }
105     return render(request, 'dashboard.html', context)
106
107
108

```

```

FinalProject > FinalApp > views.py
110 class update_fee_form(forms.ModelForm):
111     class Meta:
112         model = StudentProfile
113         fields = ['amount_paid', 'total_fee']
114
115 def update_fee(request, student_id):
116     student = get_object_or_404(StudentProfile, id=student_id)
117     if request.method == 'POST':
118         new_amount = float(request.POST.get('amount_paid'))
119         student.amount_paid = new_amount
120         student.save()
121         messages.success(request, "Fee updated successfully!")
122         return redirect('/dashboard')
123     return render(request, 'update_fee.html', {'student': student})
124
125
126 def balance_view(request):
127     if request.user.is_authenticated:
128         student = get_object_or_404(StudentProfile, user=request.user)
129         return render(request, 'balance.html', {'student': student})
130     else:
131         return redirect('/student')
132
133 def make_payment(request, student_id):
134     student = get_object_or_404(StudentProfile, id=student_id)
135
136     if request.method == 'POST':
137         amount = int(request.POST.get('amount'))
138
139         if amount <= 0:
140             messages.error(request, "Amount must be greater than 0.")
141         elif amount > student.balance():
142             messages.error(request, "You cannot pay more than the balance.")
143

```



```

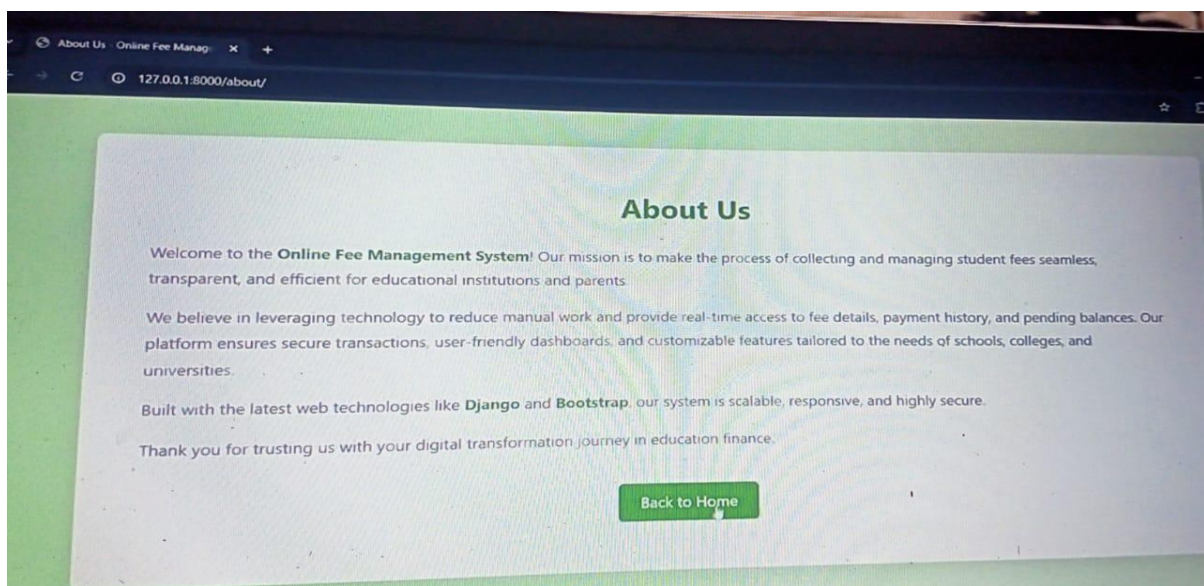
116 def update_fee(request, student_id):
117     if request.method == 'POST':
118         return redirect('/dashboard')
119     return render(request, 'update_fee.html', {'student': student})
120
121 def balance_view(request):
122     if request.user.is_authenticated:
123         student = get_object_or_404(StudentProfile, user=request.user)
124         return render(request, 'balance.html', {'student': student})
125     else:
126         return redirect('/student')
127
128 def make_payment(request, student_id):
129     student = get_object_or_404(StudentProfile, id=student_id)
130
131     if request.method == 'POST':
132         amount = int(request.POST.get('amount'))
133
134         if amount <= 0:
135             messages.error(request, "Amount must be greater than 0.")
136         elif amount > student.balance():
137             messages.error(request, "You cannot pay more than the balance.")
138         else:
139             student.amount_paid += amount
140             student.save()
141             messages.success(request, f"Payment of {amount} successful!")
142             return redirect('/balance', student_id=student.id)
143
144     return render(request, 'make_payment.html', {'student': student})

```

Navbar



About



Contact Us

Contact Us

Email: ratnamanohar036@gmail.com

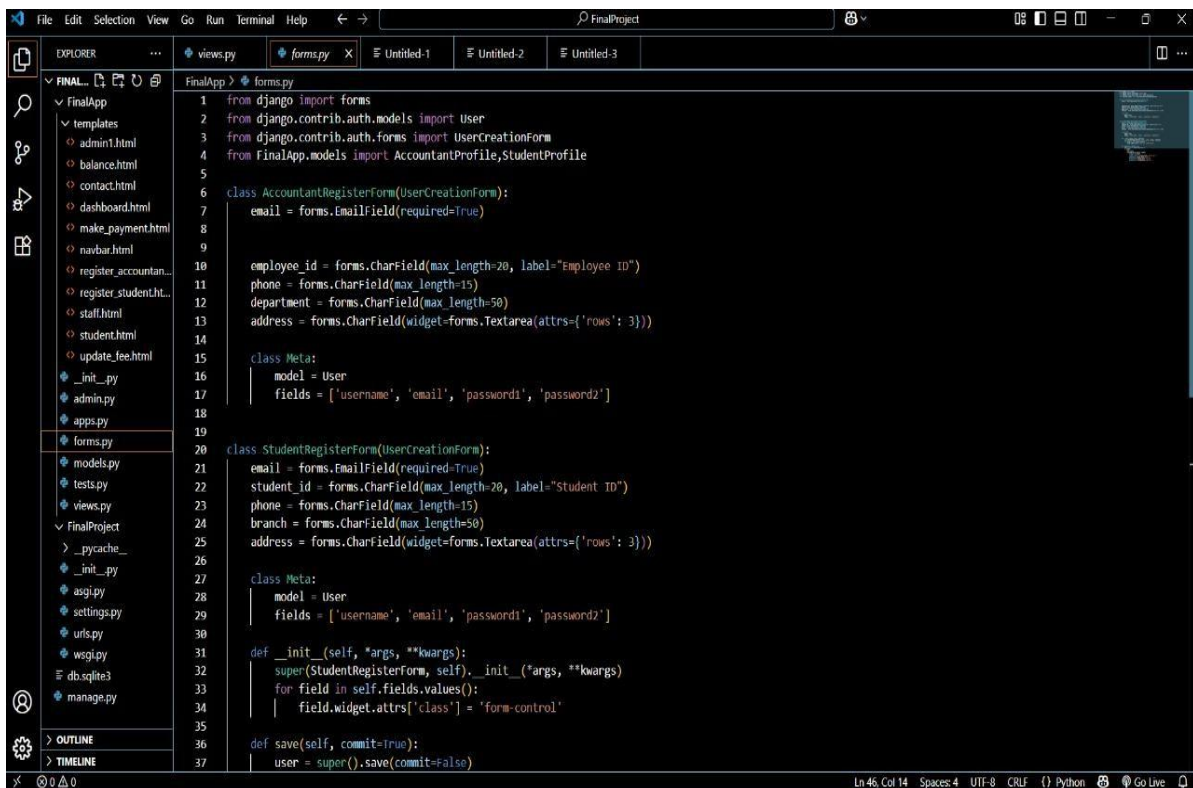
Phone: +91 98765 43210

Address: Near ArogyaMatha Hospital,
kankipadu, Krishna, AP - 521151

Working Hours: Mon - Fri, 9:00 AM to 6:00 PM

[Back to Home](#)

Forms.py



```
1 from django import forms
2 from django.contrib.auth.models import User
3 from django.contrib.auth.forms import UserCreationForm
4 from FinalApp.models import AccountantProfile, StudentProfile
5
6 class AccountantRegisterForm(UserCreationForm):
7     email = forms.EmailField(required=True)
8
9
10    employee_id = forms.CharField(max_length=20, label="Employee ID")
11    phone = forms.CharField(max_length=15)
12    department = forms.CharField(max_length=50)
13    address = forms.CharField(widget=forms.Textarea(attrs={'rows': 3}))
14
15    class Meta:
16        model = User
17        fields = ['username', 'email', 'password1', 'password2']
18
19
20 class StudentRegisterForm(UserCreationForm):
21     email = forms.EmailField(required=True)
22     student_id = forms.CharField(max_length=20, label="Student ID")
23     phone = forms.CharField(max_length=15)
24     branch = forms.CharField(max_length=50)
25     address = forms.CharField(widget=forms.Textarea(attrs={'rows': 3}))
26
27    class Meta:
28        model = User
29        fields = ['username', 'email', 'password1', 'password2']
30
31    def __init__(self, *args, **kwargs):
32        super(StudentRegisterForm, self).__init__(*args, **kwargs)
33        for field in self.fields.values():
34            field.widget.attrs['class'] = 'form-control'
35
36    def save(self, commit=True):
37        user = super().save(commit=False)
```

```

1 from django import forms
2 from django.contrib.auth.models import User
3 from django.contrib.auth.forms import UserCreationForm
4 from FinalApp.models import AccountantProfile, StudentProfile
5
6 class AccountantRegisterForm(UserCreationForm):
7     email = forms.EmailField(required=True)
8
9
10    employee_id = forms.CharField(max_length=20, label="Employee ID")
11    phone = forms.CharField(max_length=15)
12    department = forms.CharField(max_length=50)
13    address = forms.CharField(widget=forms.Textarea(attrs={'rows': 3}))
14
15    class Meta:
16        model = User
17        fields = ['username', 'email', 'password1', 'password2']
18
19
20 class StudentRegisterForm(UserCreationForm):
21     email = forms.EmailField(required=True)
22     student_id = forms.CharField(max_length=20, label="Student ID")
23     phone = forms.CharField(max_length=15)
24     branch = forms.CharField(max_length=50)
25     address = forms.CharField(widget=forms.Textarea(attrs={'rows': 3}))
26
27    class Meta:
28        model = User
29        fields = ['username', 'email', 'password1', 'password2']
30
31    def __init__(self, *args, **kwargs):
32        super(StudentRegisterForm, self).__init__(*args, **kwargs)
33        for field in self.fields.values():
34            field.widget.attrs['class'] = 'form-control'
35
36    def save(self, commit=True):
37        user = super().save(commit=False)

```

```

20 class StudentRegisterForm(UserCreationForm):
21     def __init__(self, *args, **kwargs):
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36    def save(self, commit=True):
37        user = super().save(commit=False)
38        if commit:
39            user.save()
40            StudentProfile.objects.create(
41                student_user=user,
42                student_id=self.cleaned_data['student_id'],
43                phone=self.cleaned_data['phone'],
44                branch=self.cleaned_data['branch'],
45                address=self.cleaned_data['address']
46            )

```

Admin Login

127.0.0.1:8000/admin1/

Reload this page

Admin Login

Username

Password

Login Back to Home

Accountant Registration

127.0.0.1:8000/register_accountant/

Accountant Registration

Username

Email

Password

Password confirmation

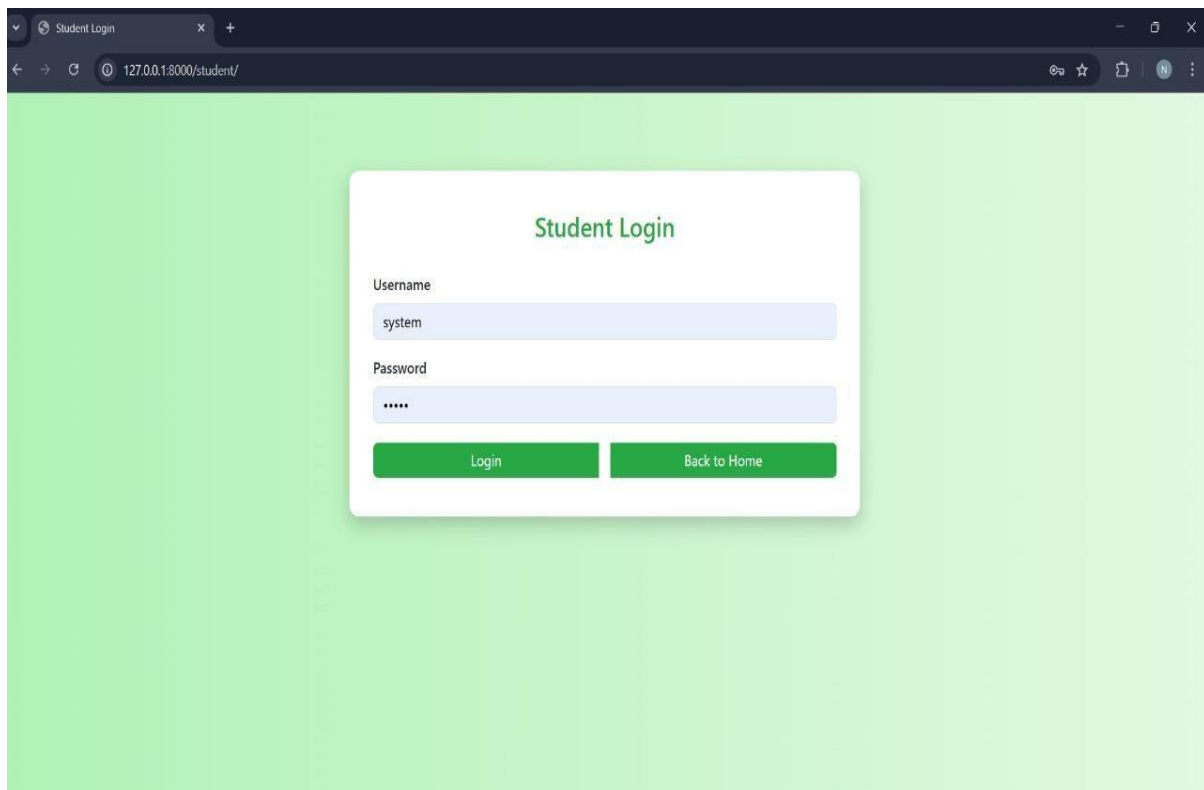
Employee ID

Phone

Department

Address

Register Back to Home



Models.py

```

File Edit Selection View Go Run Terminal Help
FinalProject

EXPLORER
FinalApp
  > __pycache__
  > migrations
  > static
  > templates
  > _init_.py
  > admin.py
  > apps.py
  > forms.py
  > models.py
  > tests.py
  > views.py
FinalProject
  > __pycache__
  > _init_.py
  > asgi.py
  > settings.py
  > urls.py
  > wsgi.py
  > db.sqlite3
  > manage.py

models.py
1 from django.db import models
2 from django.contrib.auth.models import User
3
4 class AccountantProfile(models.Model):
5     user = models.OneToOneField(User, on_delete=models.CASCADE)
6     employee_id = models.CharField(max_length=20, unique=True)
7     phone = models.CharField(max_length=15)
8     department = models.CharField(max_length=50)
9     address = models.TextField(blank=True)
10
11     def __str__(self):
12         return f'{self.user.username} ({self.employee_id})'
13
14 class StudentProfile(models.Model):
15     student_user = models.OneToOneField(User, on_delete=models.CASCADE, null=True)
16     student_id = models.CharField(max_length=20, unique=True)
17     phone = models.CharField(max_length=15)
18     branch = models.CharField(max_length=50)
19     address = models.TextField(blank=True)
20     amount_paid = models.DecimalField(max_digits=10, decimal_places=2, default=0)
21     total_fee = models.DecimalField(max_digits=10, decimal_places=2, default=25000)
22
23     def __str__(self):
24         return f'{self.student_user.username} - Student'
25
26     def balance(self):
27         return self.total_fee - self.amount_paid
  
```


Welcome, Bhavya_Sri

Student ID: TM58

Email: bhavyasri@gmail.com

Phone: 8975461464

Branch: MPC

Total Fee: ₹25000.00

Amount Paid: ₹25000.00

Balance: ₹0.00

Back to Home

Make Payment

Accountant Dashboard

Total Students: 5

Collected Amount: ₹86000

Total Balance: ₹14000

Student Fee Details

Student ID	Name	Branch	Amount Paid	Total Fee	Balance
DUMMY123		CSE	₹25000.00	₹0.00	₹-25000.00
TM58	Bhavya_Sri	MPC	₹25000.00	₹25000.00	₹0.00
TMY68	Arjun	MPC	₹0.00	₹25000.00	₹25000.00
TMY13	Srinidhi	MPC	₹18000.00	₹25000.00	₹7000.00
TMY 21	Karthik	MPC	₹18000.00	₹25000.00	₹7000.00

Staff Login

127.0.0.1:8000/staff/

Staff Login

Username

Password

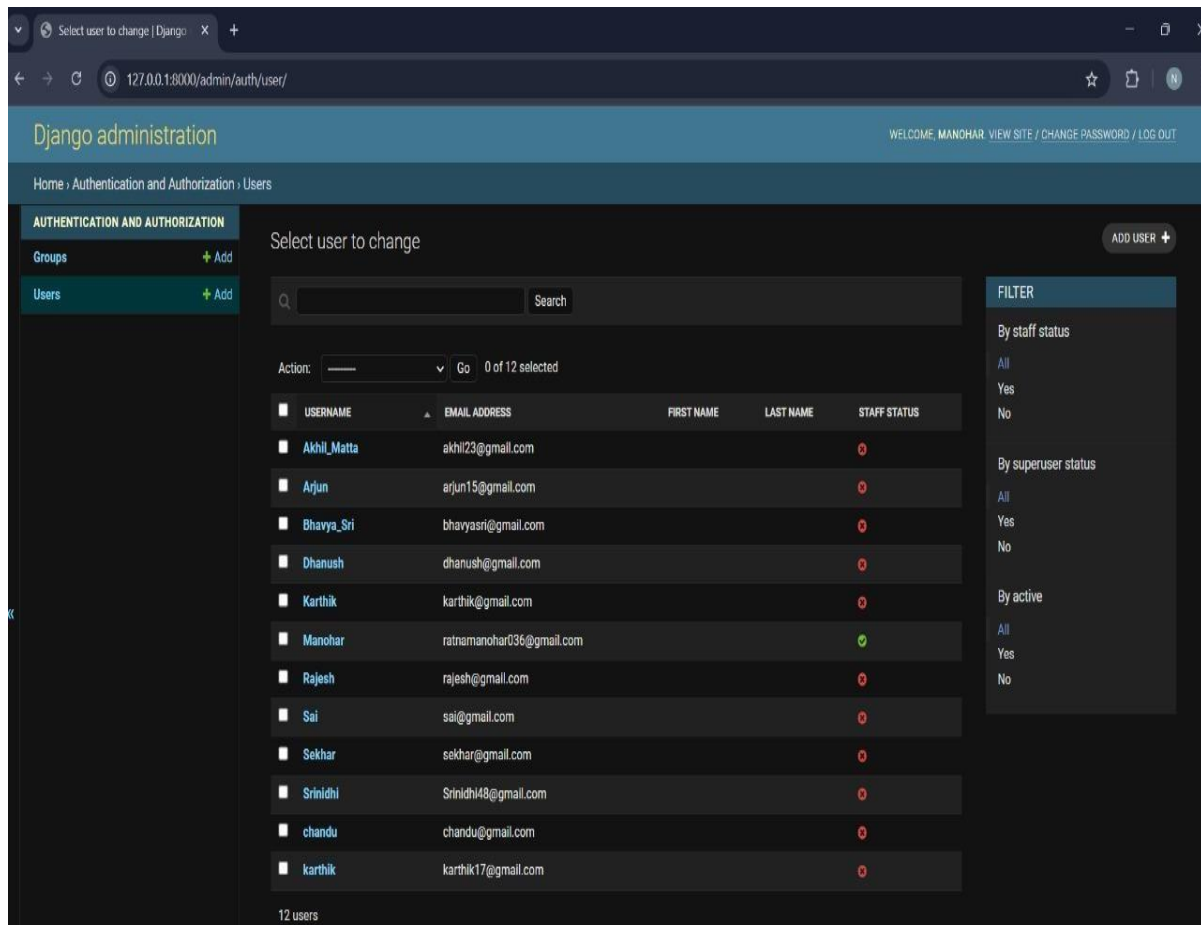
Login Back to Home

Pay Fee

Enter Amount to Pay (₹):

Pay Now Back to Home

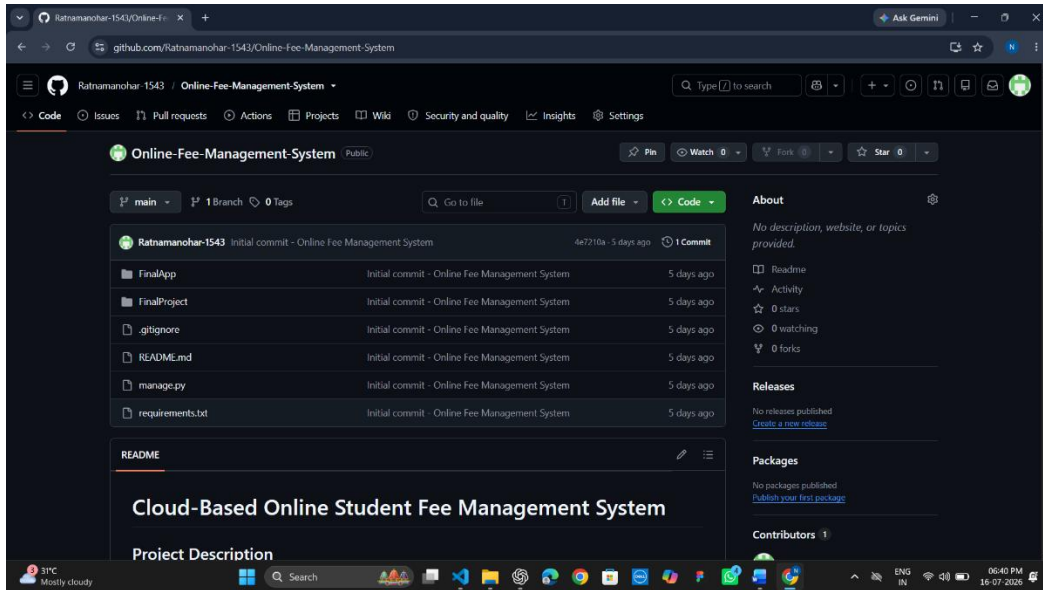
Admin.py



Commands used for Database - MYSQL:

- sudo apt update
- sudo apt upgrade -y
- sudo apt install mysql-client
- sudo apt install default-libmysqlclient-dev pkg-config
- cd Online-Fee-Management-System
- ls
- pwd
- nano FinalProject/settings.py

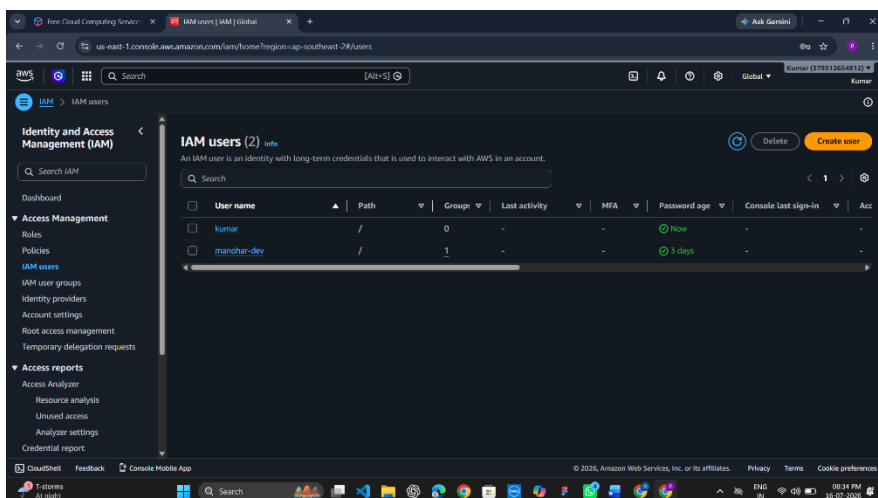
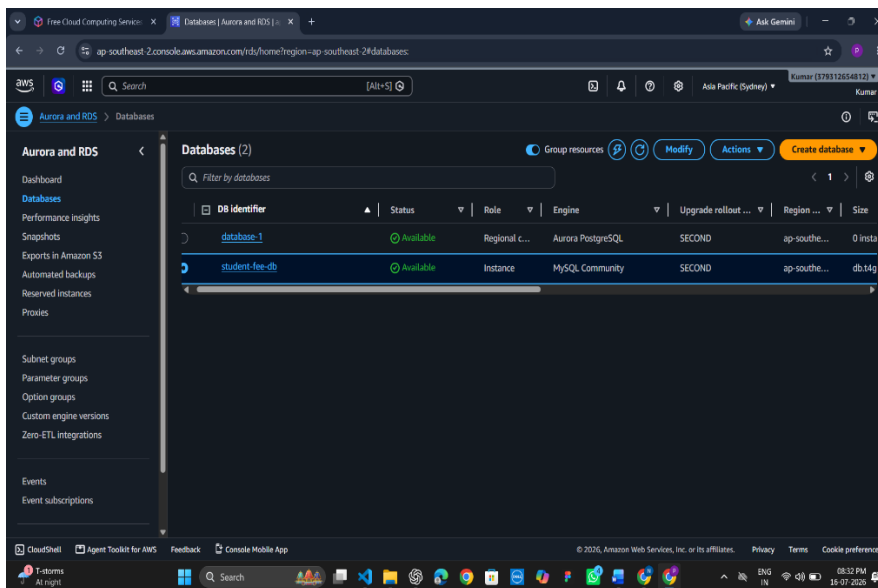
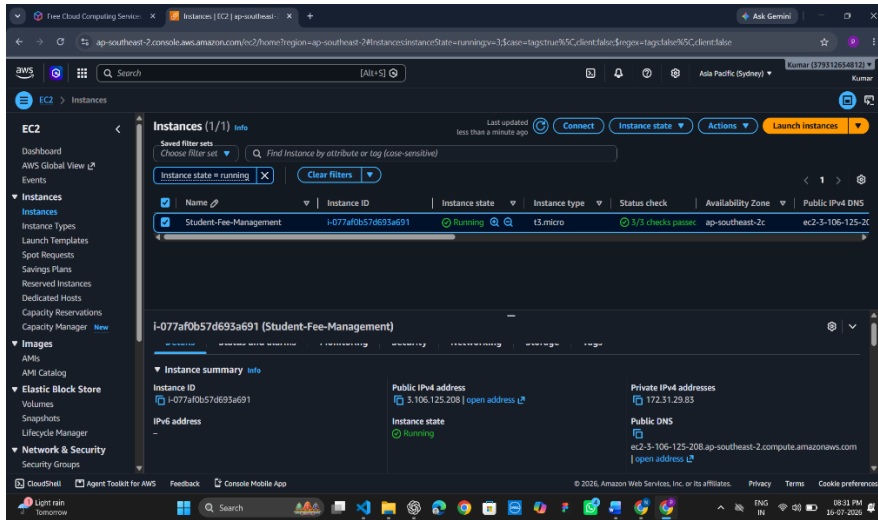
Git hub pages:

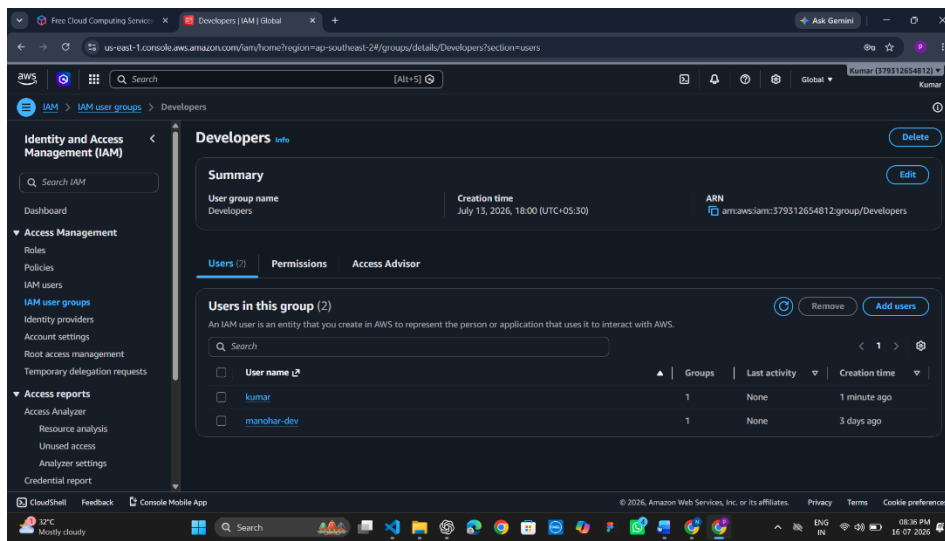


Commands Used in GitHub

- git init
- git status
- git add .
- git commit -m "Initial Commit"
- git branch -M main
- git remote add origin https://github.com/Ratnamanohar-1543/Online-Fee-Management-System.git
- git push -u origin main
- git clone https://github.com/Ratnamanohar-1543/Online-Fee-Management-System.git
- git pull origin main

AWS Pages:





CONCLUSION

The **Cloud-Based Student Fee Management System** developed using the Django framework and deployed on **Amazon Web Services (AWS)** provides an efficient, secure, and scalable solution for managing student fee records. By integrating **Amazon EC2** for application hosting, **Amazon RDS MySQL** for database management, and **AWS IAM** for secure access control, the system successfully demonstrates the practical implementation of cloud computing concepts in an educational environment.

The application simplifies fee management by enabling administrators and accountants to maintain student information, update fee payments, and monitor fee balances through a centralized web-based platform. Students can securely access their fee details, improving transparency and reducing the dependency on manual record-keeping. The use of GitHub for version control ensures efficient source code management, easy collaboration, and reliable deployment to the cloud.

Deploying the project on AWS enhances its availability, reliability, and scalability compared to a traditional local application. The implementation of IAM improves security by providing controlled access to AWS resources, while Amazon RDS offers a managed and reliable database solution for storing financial records. Overall, this project demonstrates how cloud technologies can modernize educational management systems by improving operational efficiency, data security, and accessibility. It also provides a strong foundation for future enhancements such as automated deployment, cloud storage, monitoring, and additional DevOps practices, making it a valuable learning experience in AWS Cloud and DevOps.